

BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO MÔN HỌC
HỆ THỐNG KHUYẾN NGHỊ
(Recommender Systems)

Chủ Đề 9
HỆ THỐNG KHUYẾN NGHỊ
DỰA TRÊN PHIÊN DUYỆT WEB
(Session-based Recommendation System)

Giảng viên hướng dẫn: TS. Lê Thị Vĩnh Thanh

Chủ đề thực hiện: Chủ đề 9

Thành viên:

1. Phan Nguyễn Mai Phương – MSHV: 25002711
2. Ngô Quốc Hoàng – MSHV: 25001771
3. Bùi Thức Nam – MSHV: 25003881

TP. Hồ Chí Minh, Ngày 28 tháng 5 năm 2026

MỤC LỤC

1	GIỚI THIỆU	1
1.1	Bối cảnh và động cơ nghiên cứu	1
1.2	Bài toán thực tế	1
1.3	Định nghĩa hình thức bài toán khuyến nghị theo phiên	2
1.4	So sánh chi tiết với lọc cộng tác truyền thống	2
1.5	Mục tiêu và phạm vi báo cáo	3
2	CƠ SỞ LÝ THUYẾT	5
2.1	Hệ thống khuyến nghị tổng quan	5
2.1.1	Collaborative Filtering (Lọc cộng tác)	5
2.1.2	Content-based Filtering (Lọc dựa trên nội dung)	5
2.1.3	Phương pháp lai (Hybrid Methods)	6
2.2	Session và cấu trúc dữ liệu session	6
2.2.1	Định nghĩa session	6
2.2.2	Định dạng event log	6
2.2.3	Đặc điểm thống kê của dữ liệu session	7
2.3	Tiền xử lý dữ liệu session	7
2.3.1	Lọc item hiếm (Rare item filtering)	7
2.3.2	Lọc session ngắn và dài	8
2.3.3	Chia dữ liệu theo thời gian (Time-based split)	8
2.3.4	Quy trình tiền xử lý tổng hợp	8
2.4	Tạo cặp huấn luyện	9
2.4.1	Chiến lược Sliding Window	9
2.4.2	Chiến lược Leave-one-out (cho đánh giá)	9
2.5	Thuật toán SKNN (Session-based K-Nearest Neighbors)	9
2.5.1	Ý tưởng chính	9
2.5.2	Ký hiệu	10
2.5.3	Công thức độ tương tự Cosine trên vector nhị phân	10
2.5.4	Công thức tính điểm	11
2.5.5	Mã giả thuật toán SKNN	11
2.5.6	Tối ưu hóa bằng chỉ mục đảo (Inverted Index)	12
2.5.7	Phân tích độ phức tạp	13
2.6	Thuật toán GRU4Rec	13
2.6.1	Mạng hồi quy RNN và vấn đề tiêu biến gradient	13
2.6.2	Gated Recurrent Unit (GRU)	14
2.6.3	Kiến trúc GRU4Rec	15

2.6.4	Hàm mất mát	16
2.6.5	Mã giả huấn luyện GRU4Rec	17
2.6.6	Mã giả dự đoán GRU4Rec	18
2.6.7	Phân tích độ phức tạp GRU4Rec	19
2.7	Các chỉ số đánh giá	20
2.7.1	Recall@K (Hit Rate)	20
2.7.2	MRR@K (Mean Reciprocal Rank)	20
2.7.3	HR@K (Hit Rate)	21
2.7.4	NDCG@K (Normalized Discounted Cumulative Gain)	21
3	THÍ NGHIỆM	22
3.1	Bộ dữ liệu Yoochoose (RecSys Challenge 2015)	22
3.1.1	Giới thiệu bộ dữ liệu	22
3.1.2	Cấu trúc dữ liệu	22
3.1.3	Thống kê tổng quan	23
3.1.4	Lý do lấy mẫu 1/64	23
3.2	Cài đặt thí nghiệm	24
3.2.1	Siêu tham số (hyperparameters)	24
3.2.2	Phần cứng và phần mềm	25
3.3	Tiền xử lý cụ thể	25
3.4	Giao thức đánh giá và tính hợp lệ của kết quả	26
3.5	Kết quả mô hình Popularity Baseline	27
3.6	Kết quả SKNN (K=500)	27
3.7	Kết quả GRU4Rec (10 epoch)	28
3.8	Thử nghiệm K khác nhau cho SKNN	28
3.9	Đường cong loss huấn luyện của GRU4Rec	30
4	PHÂN TÍCH VÀ THẢO LUẬN	32
4.1	So sánh tổng hợp 3 mô hình	32
4.2	Phân tích tại sao SKNN vượt Popularity (+27.65 điểm)	32
4.2.1	Tận dụng thông tin phiên hiện tại	33
4.2.2	Khai thác co-occurrence pattern	33
4.2.3	Phân tích định lượng	33
4.3	Phân tích tại sao GRU4Rec chưa vượt SKNN	33
4.3.1	Học được thứ tự (Sequential patterns)	34
4.3.2	Biểu diễn liên tục (Continuous representation)	34
4.3.3	Tại sao GRU4Rec lại thấp hơn SKNN ở đây?	34
4.4	Ưu nhược điểm từng phương pháp	35
4.5	Ảnh hưởng của siêu tham số K	36
4.5.1	Cơ chế ảnh hưởng	36

4.5.2	Phân tích theo lý thuyết bias–variance	36
4.5.3	Khuyến nghị chọn K trong thực tế	36
4.6	Hạn chế của thí nghiệm	36
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	38
5.1	Tóm tắt kết quả	38
5.2	Đóng góp chính	38
5.3	Hướng mở rộng	39
5.3.1	NARM – Neural Attentive Recommendation Machine	39
5.3.2	SR-GNN – Session-based Recommendation with Graph Neural Net- works	39
5.3.3	BERT4Rec – Bidirectional Encoder Representations for Recommen- dation	40
5.3.4	Dataset tiếng Việt	40
5.3.5	Các hướng khác	41
	PHỤ LỤC	44
A.	Hướng dẫn chạy lại thí nghiệm	44
B.	Trích đoạn cốt lõi của SKNN	44
C.	Trích đoạn cốt lõi của GRU4Rec và đánh giá	45

DANH MỤC HÌNH VẼ

3.1	Ảnh hưởng của K đến Recall@20 của SKNN	29
3.2	Đường cong loss huấn luyện của GRU4Rec qua 10 epoch	30
4.1	Biểu đồ so sánh Recall@20 và MRR@20 của 3 mô hình	32

DANH MỤC BẢNG

1.1	So sánh lọc cộng tác truyền thống và khuyến nghị theo phiên	3
2.1	Ví dụ cấu trúc dữ liệu event log	7
2.2	Ví dụ tạo cặp huấn luyện từ session $[A, B, C, D, E]$	9
2.3	Bảng ký hiệu thuật toán SKNN	10
2.4	So sánh các độ đo tương tự cho session	11
2.5	Giải thích các thành phần trong GRU	15
2.6	So sánh Cross-Entropy Loss và BPR Loss	17
2.7	So sánh các chỉ số đánh giá	21
3.1	Thống kê bộ dữ liệu Yoochoose	23
3.2	Cấu hình hyperparameters cho các mô hình	24
3.3	Môi trường thí nghiệm	25
3.4	Số liệu qua từng bước tiền xử lý	25
3.5	Kết quả Popularity Baseline	27
3.6	Kết quả SKNN với $K=500$	27
3.7	Kết quả GRU4Rec sau 10 epoch huấn luyện	28
3.8	Ảnh hưởng của K đến hiệu năng SKNN	29
4.1	Bảng so sánh tổng hợp kết quả 3 mô hình	32
4.2	Phân tích chi tiết sự cải thiện của SKNN so với Popularity	33
4.3	Phân tích ưu nhược điểm chi tiết	35
5.1	Tổng hợp các hướng mở rộng	41
P.1	Các tệp quan trọng để tái lập kết quả	44

1. GIỚI THIỆU

1.1. Bối cảnh và động cơ nghiên cứu

Trong thời đại thương mại điện tử phát triển mạnh mẽ, các hệ thống khuyến nghị (Recommender Systems) đóng vai trò then chốt trong việc cá nhân hóa trải nghiệm người dùng [9, 10]. Tuy nhiên, các phương pháp truyền thống như Collaborative Filtering (CF) hay Content-based Filtering thường cần nhận diện người dùng thông qua tài khoản đăng nhập và lịch sử tương tác dài hạn.

Trong nhiều ứng dụng thực tế, người dùng có thể truy cập hệ thống mà **không đăng nhập**; khi đó, hệ thống chỉ quan sát được chuỗi hành động trong phiên hiện tại (current session) [1, 2]. Đối với nhóm người dùng này, các phương pháp CF truyền thống gặp khó khăn vì không có hồ sơ dài hạn để so sánh hoặc phân rã ma trận người dùng-item.

Bên cạnh đó, ngay cả với người dùng đã đăng nhập, sở thích của họ có thể thay đổi theo thời gian và ngữ cảnh. Một người dùng có thể tìm kiếm quần áo thể thao vào buổi sáng nhưng lại quan tâm đến sách kỹ thuật vào buổi tối. Lịch sử dài hạn không phản ánh chính xác **ý định tức thời** (immediate intent) của người dùng tại thời điểm hiện tại.

Từ những thách thức trên, bài toán **Session-based Recommendation** (Khuyến nghị dựa trên phiên) ra đời nhằm giải quyết vấn đề: *Làm thế nào để gợi ý sản phẩm phù hợp cho người dùng chỉ dựa vào chuỗi hành động ngắn trong phiên hiện tại, mà không cần bất kỳ thông tin định danh hay lịch sử dài hạn nào?*

1.2. Bài toán thực tế

Để hiểu rõ hơn bài toán, hãy xem xét tình huống thực tế sau:

Tình huống: Một khách hàng ẩn danh truy cập trang web bán hàng Shopee. Trong vòng 10 phút, khách hàng lần lượt xem các sản phẩm:

Áo thun nam → Quần jean slim → Giày sneaker → **Gợi ý tiếp theo**

Câu hỏi: Hệ thống nên gợi ý sản phẩm nào tiếp theo? Có thể là thắt lưng, tất, hay áo khoác – những sản phẩm thường được mua cùng với bộ trang phục nam giới.

Bài toán này xuất hiện phổ biến trong nhiều lĩnh vực:

- **Thương mại điện tử:** Amazon, Shopee, Tiki – gợi ý sản phẩm cho khách vắng lại
- **Trang tin tức:** VnExpress, Tuổi Trẻ – gợi ý bài viết tiếp theo
- **Streaming video:** YouTube, Netflix – gợi ý video tiếp theo
- **Âm nhạc:** Spotify, Zing MP3 – gợi ý bài hát tiếp theo trong phiên nghe

- **Du lịch:** Booking.com, Traveloka – gợi ý khách sạn/chuyến bay

Đặc điểm chung của các ứng dụng trên là: hệ thống cần đưa ra khuyến nghị **real-time** (thời gian thực), chỉ dựa vào một chuỗi ngắn các hành động gần đây, mà không cần biết người dùng là ai.

1.3. Định nghĩa hình thức bài toán khuyến nghị theo phiên

Định nghĩa 1.1 (Session). Một phiên (session) $S = [i_1, i_2, \dots, i_t]$ là một chuỗi có thứ tự các item mà một người dùng ẩn danh đã tương tác (click, xem, thêm vào giỏ hàng) trong một khoảng thời gian liên tục. Phiên kết thúc khi người dùng không hoạt động quá một ngưỡng thời gian (thường là 30 phút) hoặc khi đóng trình duyệt.

Định nghĩa 1.2 (Bài toán Session-based Recommendation). Cho:

- Tập item $\mathcal{I} = \{i_1, i_2, \dots, i_M\}$ với M là tổng số item trong hệ thống
- Phiên truy vấn hiện tại $S_q = [i_1, i_2, \dots, i_t]$ với t item đã tương tác

Mục tiêu: Xây dựng hàm $f : S_q \rightarrow \mathbb{R}^M$ sao cho $f(S_q)$ trả về vector điểm số (score) cho tất cả M item, từ đó chọn ra Top- N item có điểm cao nhất làm khuyến nghị:

$$\text{Top-}N = \underset{i \notin S_q}{\operatorname{argmax-}N} f(S_q)[i] \quad (1.1)$$

Điểm khác biệt quan trọng so với bài toán khuyến nghị truyền thống:

1. **Không có User ID:** Mỗi phiên là độc lập, không liên kết với bất kỳ hồ sơ người dùng nào
2. **Dữ liệu ngắn hạn:** Chỉ có vài item (trung bình 3–5 item) trong phiên hiện tại
3. **Thứ tự quan trọng:** Chuỗi $[A, B, C]$ khác với $[C, B, A]$ – thứ tự click phản ánh ý định
4. **Yêu cầu real-time:** Khuyến nghị phải được tính trong vài mili-giây

1.4. So sánh chi tiết với lọc cộng tác truyền thống

Bảng 1.1 trình bày so sánh chi tiết giữa phương pháp Collaborative Filtering truyền thống và Session-based Recommendation trên nhiều tiêu chí khác nhau.

Bảng 1.1: So sánh lọc cộng tác truyền thống và khuyến nghị theo phiên

Tiêu chí	CF truyền thống	Session-based RS
Yêu cầu User ID	Bắt buộc	Không cần
Dữ liệu đầu vào	Ma trận User-Item (rating hoặc implicit feedback dài hạn)	Chuỗi item trong phiên hiện tại
Xử lý cold-start user	Không giải quyết được	Xử lý tốt (mọi phiên đều là “mới”)
Nắm bắt sở thích tức thời	Kém (dùng lịch sử tổng hợp)	Tốt (chỉ dùng hành vi hiện tại)
Thứ tự tương tác	Bỏ qua (chỉ quan tâm có/không)	Quan trọng (chuỗi có thứ tự)
Khả năng mở rộng	Cần cập nhật ma trận khi có user mới	Không cần – mỗi phiên xử lý độc lập
Bảo mật thông tin	Lưu trữ lịch sử cá nhân	Không lưu thông tin cá nhân
Ứng dụng điển hình	Netflix, Spotify (user đăng nhập)	Amazon, Shopee (khách vắng lai)
Phương pháp tiêu biểu	Matrix Factorization, SVD, ALS	SKNN, GRU4Rec, NARM, SR-GNN

Từ bảng so sánh trên, có thể thấy Session-based RS không phải là sự thay thế hoàn toàn cho CF truyền thống, mà là một **bổ sung quan trọng** cho các tình huống mà CF không thể áp dụng. Trong thực tế, nhiều hệ thống kết hợp cả hai: dùng CF cho người dùng đã đăng nhập và Session-based RS cho khách vắng lai.

1.5. Mục tiêu và phạm vi báo cáo

Báo cáo này tập trung vào việc nghiên cứu, triển khai và đánh giá hai phương pháp tiêu biểu cho bài toán Session-based Recommendation:

1. **Session-based K-Nearest Neighbors (SKNN):** Phương pháp dựa trên hàng xóm gần nhất, đơn giản nhưng hiệu quả cao, được đề xuất và phân tích bởi Ludewig & Jannach (2018) [2].
2. **GRU4Rec:** Phương pháp deep learning sử dụng mạng hồi quy GRU (Gated Recurrent Unit), được đề xuất bởi Hidasi et al. (2016) [1] – bài báo tiên phong trong

việc áp dụng RNN cho session-based recommendation.

Phạm vi báo cáo:

- Trình bày cơ sở lý thuyết chi tiết của cả hai phương pháp
- Triển khai thực nghiệm trên bộ dữ liệu Yoochoose (RecSys Challenge 2015)
- So sánh hiệu năng với Popularity Baseline
- Phân tích ảnh hưởng của các siêu tham số (hyperparameters)
- Thảo luận ưu nhược điểm và hướng phát triển

2. CƠ SỞ LÝ THUYẾT

2.1. Hệ thống khuyến nghị tổng quan

Hệ thống khuyến nghị (Recommender System) là một lớp các hệ thống lọc thông tin nhằm dự đoán “đánh giá” hoặc “sở thích” mà người dùng sẽ dành cho một item. Có ba hướng tiếp cận chính:

2.1.1. Collaborative Filtering (Lọc cộng tác)

Collaborative Filtering (CF) dựa trên giả thuyết: “*Những người dùng có hành vi tương tự trong quá khứ sẽ có sở thích tương tự trong tương lai.*” Đây là nhóm phương pháp nền tảng trong hệ thống khuyến nghị truyền thống [9, 10].

CF được chia thành hai loại chính:

a) User-based CF: Tìm những người dùng có hành vi tương tự với người dùng mục tiêu, sau đó gợi ý các item mà những người dùng tương tự đã thích nhưng người dùng mục tiêu chưa biết.

Công thức dự đoán rating:

$$\hat{r}_{u,i} = \bar{r}_u + \frac{\sum_{v \in N(u)} \text{sim}(u, v) \cdot (r_{v,i} - \bar{r}_v)}{\sum_{v \in N(u)} |\text{sim}(u, v)|} \quad (2.1)$$

trong đó $N(u)$ là tập hàng xóm của user u , $\text{sim}(u, v)$ là độ tương tự giữa user u và v , $\bar{r}_u$ là rating trung bình của user u .

b) Item-based CF: Tìm những item tương tự với item mà người dùng đã thích, dựa trên pattern đánh giá của tất cả người dùng.

$$\hat{r}_{u,i} = \frac{\sum_{j \in N(i)} \text{sim}(i, j) \cdot r_{u,j}}{\sum_{j \in N(i)} |\text{sim}(i, j)|} \quad (2.2)$$

Hạn chế của CF:

- Cold-start problem: Không thể gợi ý cho user mới hoặc item mới
- Sparsity: Ma trận User-Item thường rất thưa (>99% giá trị trống)
- Yêu cầu User ID: Không áp dụng được cho người dùng ẩn danh

2.1.2. Content-based Filtering (Lọc dựa trên nội dung)

Content-based Filtering gợi ý item dựa trên đặc trưng nội dung (features) của item và hồ sơ sở thích (user profile) được xây dựng từ lịch sử tương tác.

Ý tưởng: Nếu người dùng đã thích item có đặc trưng f , hệ thống sẽ gợi ý các item khác cũng có đặc trưng f .

Ưu điểm: Không cần dữ liệu từ người dùng khác, giải quyết được cold-start item (nếu có metadata).

Nhược điểm: Bị giới hạn trong “bong bóng” nội dung (filter bubble), không khám phá được item mới lạ.

2.1.3. Phương pháp lai (Hybrid Methods)

Kết hợp CF và Content-based để tận dụng ưu điểm của cả hai. Các chiến lược kết hợp phổ biến:

- **Weighted:** Kết hợp điểm số từ nhiều phương pháp với trọng số
- **Switching:** Chuyển đổi giữa các phương pháp tùy tình huống
- **Feature combination:** Kết hợp đặc trưng từ nhiều nguồn
- **Cascade:** Dùng phương pháp này lọc trước, phương pháp kia tinh chỉnh sau

2.2. Session và cấu trúc dữ liệu session

2.2.1. Định nghĩa session

Trong ngữ cảnh web, một **session** (phiên) là một chuỗi các tương tác liên tiếp của cùng một người dùng (hoặc cùng một trình duyệt) trong một khoảng thời gian giới hạn. Session thường được xác định bởi:

- **Session ID:** Mã định danh duy nhất, thường được tạo từ cookie hoặc fingerprint trình duyệt
- **Timeout:** Phiên kết thúc nếu không có hoạt động trong 30 phút (ngưỡng phổ biến)
- **Explicit end:** Đóng trình duyệt, đăng xuất, hoặc hoàn tất mua hàng

2.2.2. Định dạng event log

Dữ liệu session thường được lưu trữ dưới dạng event log với cấu trúc như Bảng 2.1.

Bảng 2.1: Ví dụ cấu trúc dữ liệu event log

Session ID	Timestamp	Item ID	Action	Category
S001	2015-04-01 09:01:03	214530	click	Electronics
S001	2015-04-01 09:03:45	214531	click	Electronics
S001	2015-04-01 09:07:12	214533	click	Accessories
S001	2015-04-01 09:09:55	214530	buy	Electronics
S002	2015-04-01 10:15:00	312076	click	Fashion
S002	2015-04-01 10:18:30	312080	click	Fashion
S002	2015-04-01 10:22:10	312076	click	Fashion

Từ event log, ta trích xuất chuỗi item cho mỗi session:

- Session S001: [214530, 214531, 214533, 214530]
- Session S002: [312076, 312080, 312076]

2.2.3. Đặc điểm thống kê của dữ liệu session

Dữ liệu session có một số đặc điểm thống kê quan trọng:

1. **Phân phối độ dài session:** Tuân theo phân phối lệch phải (right-skewed), đa số session rất ngắn (2–5 item), một số ít session rất dài (>50 item)
2. **Phân phối tần suất item:** Tuân theo luật lũy thừa (power law) – một số ít item rất phổ biến, đa số item hiếm khi xuất hiện
3. **Tính thời gian:** Hành vi người dùng thay đổi theo thời gian (trend, seasonality)
4. **Tính lặp lại:** Trong cùng một session, người dùng có thể quay lại xem item đã xem trước đó

2.3. Tiền xử lý dữ liệu session

Tiền xử lý dữ liệu là bước quan trọng ảnh hưởng trực tiếp đến chất lượng mô hình. Các bước tiền xử lý chính bao gồm:

2.3.1. Lọc item hiếm (Rare item filtering)

Item xuất hiện quá ít lần (ví dụ dưới 5 lần trong toàn bộ dataset) không cung cấp đủ thông tin thống kê để học pattern. Việc giữ lại chúng có thể gây nhiễu và tăng kích thước vocabulary không cần thiết.

$$\mathcal{I}_{filtered} = \{i \in \mathcal{I} \mid \text{count}(i) \geq \tau_{min}\} \quad (2.3)$$

trong đó τ_{min} là ngưỡng tần suất tối thiểu (thường $\tau_{min} = 5$).

2.3.2. Lọc session ngắn và dài

Session quá ngắn (chỉ 1 item): Không thể tạo cặp huấn luyện (input, label) vì cần ít nhất 2 item.

Session quá dài (>20–50 item): Có thể là bot, crawler, hoặc session bị ghép nhầm. Những session này không đại diện cho hành vi người dùng bình thường.

$$\mathcal{S}_{filtered} = \{S \in \mathcal{S} \mid l_{min} \leq |S| \leq l_{max}\} \quad (2.4)$$

Trong thí nghiệm của chúng tôi: $l_{min} = 2$, $l_{max} = 20$.

2.3.3. Chia dữ liệu theo thời gian (Time-based split)

Khác với chia ngẫu nhiên (random split), chia theo thời gian đảm bảo tính thực tế: mô hình chỉ được huấn luyện trên dữ liệu quá khứ và đánh giá trên dữ liệu tương lai.

- **Training set:** Các session xảy ra trước thời điểm T_{split}
- **Test set:** Các session xảy ra sau thời điểm T_{split}

Nếu dùng random split, mô hình có thể “nhìn thấy” thông tin từ tương lai (data leakage), dẫn đến đánh giá quá lạc quan.

2.3.4. Quy trình tiền xử lý tổng hợp

Quy trình tiền xử lý được tóm tắt trong Thuật toán 1.

Thuật toán 1: Tiền xử lý dữ liệu session

Input: Raw event log $\mathcal{D}$, thresholds τ_{min} , l_{min} , l_{max} , split ratio ρ

Output: Training sessions $\mathcal{S}_{train}$, Test sessions $\mathcal{S}_{test}$

- Bước 1:** Sắp xếp $\mathcal{D}$ theo (SessionID, Timestamp);
 - Bước 2:** Nhóm theo SessionID $\rightarrow$ tạo chuỗi item cho mỗi session;
 - Bước 3:** Sắp xếp session theo thời gian bắt đầu;
 - Bước 4:** Chia theo thời gian: $\mathcal{S}_{train} \leftarrow \rho \times 100\%$ session sớm nhất, $\mathcal{S}_{test} \leftarrow$ phần còn lại;
 - Bước 5:** Đếm tần suất item chỉ trên $\mathcal{S}_{train}$: $\text{count}_{train}(i)$;
 - Bước 6:** Tạo tập item hợp lệ $\mathcal{I}_{train} = \{i \mid \text{count}_{train}(i) \geq \tau_{min}\}$;
 - Bước 7:** Loại item ngoài $\mathcal{I}_{train}$ khỏi cả train và test;
 - Bước 8:** Lọc session: giữ lại session có $l_{min} \leq |S| \leq l_{max}$;
 - return** $\mathcal{S}_{train}$, $\mathcal{S}_{test}$
-

2.4. Tạo cặp huấn luyện

Từ mỗi session, ta cần tạo các cặp (input sequence, target item) để huấn luyện mô hình. Có hai chiến lược chính:

2.4.1. Chiến lược Sliding Window

Với mỗi session $S = [i_1, i_2, \dots, i_T]$, tạo $T - 1$ cặp huấn luyện:

Bảng 2.2: Ví dụ tạo cặp huấn luyện từ session $[A, B, C, D, E]$

Bước t	Input sequence	Target (label)
1	$[A]$	B
2	$[A, B]$	C
3	$[A, B, C]$	D
4	$[A, B, C, D]$	E

Chiến lược này tận dụng tối đa dữ liệu: mỗi session có T item sẽ tạo ra $T - 1$ mẫu huấn luyện.

2.4.2. Chiến lược Leave-one-out (cho đánh giá)

Khi đánh giá, ta chỉ dùng cặp cuối cùng:

- Input: $[i_1, i_2, \dots, i_{T-1}]$ (tất cả trừ item cuối)
- Label: i_T (item cuối cùng)

Đây là chiến lược đánh giá phổ biến nhất trong các nghiên cứu về session-based RS [2, 1].

2.5. Thuật toán SKNN (Session-based K-Nearest Neighbors)

2.5.1. Ý tưởng chính

Session-based K-Nearest Neighbors (SKNN) là phương pháp dựa trên nguyên lý hàng xóm gần nhất, được nghiên cứu và đánh giá chi tiết bởi Ludewig & Jannach (2018) [2] và Jannach et al. (2017) [3]. Ý tưởng cốt lõi:

“Tìm K phiên lịch sử tương tự nhất với phiên hiện tại, sau đó tổng hợp các item từ những phiên đó (có trọng số theo độ tương tự) để sinh khuyến nghị.”

Phương pháp này đơn giản về mặt khái niệm nhưng đã được chứng minh là rất hiệu quả, thậm chí vượt trội hơn nhiều phương pháp deep learning phức tạp trong một số trường hợp [2].

2.5.2. Ký hiệu

Bảng 2.3: Bảng ký hiệu thuật toán SKNN

Ký hiệu	Ý nghĩa
$S_q = [i_1, i_2, \dots, i_t]$	Phiên truy vấn (phiên hiện tại cần gợi ý)
$\mathcal{S} = \{S_1, S_2, \dots, S_N\}$	Tập tất cả phiên lịch sử (training set)
N	Tổng số phiên lịch sử
K	Số hàng xóm gần nhất (hyperparameter)
$KNN(S_q)$	Tập K phiên có similarity cao nhất với S_q
$\text{sim}(S_q, S_n)$	Độ tương tự giữa phiên S_q và S_n
$\text{score}(i)$	Điểm khuyến nghị cho item i

2.5.3. Công thức độ tương tự Cosine trên vector nhị phân

Độ tương tự giữa hai phiên được tính bằng Cosine similarity trên biểu diễn tập item nhị phân, coi mỗi phiên như một tập hợp item (bỏ qua thứ tự và trùng lặp):

$$\text{sim}(S_q, S_n) = \frac{|S_q \cap S_n|}{\sqrt{|S_q| \times |S_n|}} \quad (2.5)$$

trong đó:

- $|S_q \cap S_n|$: Số lượng item xuất hiện trong cả hai phiên (phần giao)
- $|S_q|$: Số lượng item duy nhất trong phiên truy vấn
- $|S_n|$: Số lượng item duy nhất trong phiên lịch sử S_n
- Giá trị $\text{sim} \in [0, 1]$: bằng 0 khi không có item chung, bằng 1 khi hai phiên giống hệt nhau

Nhận xét: Công thức này chính là Cosine similarity khi biểu diễn mỗi phiên dưới dạng vector nhị phân (binary vector) trong không gian M chiều (với M là tổng số item). So với Jaccard, mẫu số của Cosine binary dùng trung bình nhân kích thước hai phiên nên ít phạt phiên dài hơn.

So sánh với các độ đo khác:

Bảng 2.4: So sánh các độ đo tương tự cho session

Độ đo	Công thức	Đặc điểm
Jaccard	$\frac{ A \cap B }{ A \cup B }$	Phạt nặng session dài
Cosine (binary)	$\frac{ A \cap B }{\sqrt{ A \cdot B }}$	Cân bằng hơn
Dice	$\frac{2 A \cap B }{ A + B }$	Tương tự Jaccard

2.5.4. Công thức tính điểm

Sau khi tìm được K phiên hàng xóm gần nhất, điểm khuyến nghị cho mỗi item ứng viên i (item chưa xuất hiện trong phiên hiện tại) được tính:

$$\text{score}(i) = \sum_{S_n \in \text{KNN}(S_q)} \text{sim}(S_q, S_n) \times \mathbb{I}[i \in S_n] \quad (2.6)$$

trong đó:

- $\text{KNN}(S_q)$: Tập K phiên có similarity cao nhất với S_q
- $\mathbb{I}[i \in S_n]$: Hàm chỉ thị (indicator function), bằng 1 nếu item i có trong phiên S_n , bằng 0 nếu không
- Ý nghĩa: Item i được “bình chọn” bởi mỗi phiên hàng xóm chứa nó, với trọng số là độ tương tự của phiên đó với phiên hiện tại

Cuối cùng, khuyến nghị Top- N item có score cao nhất:

$$\text{Top-}N = \underset{i \notin S_q}{\text{argmax-}N} \text{ score}(i) \quad (2.7)$$

2.5.5. Mã giả thuật toán SKNN

Thuật toán 2 trình bày chi tiết quy trình dự đoán của SKNN.

Thuật toán 2: SKNN – dự đoán bằng hàng xóm phiên gần nhất

Input: Phiên hiện tại S_q , Tập phiên lịch sử $\mathcal{S}$, Số hàng xóm K , Số khuyến nghị N **Output:** Danh sách Top- N item khuyến nghị

```

// Bước 1: Tính similarity với tất cả phiên lịch sử
1 similarities  $\leftarrow []$ ;
2 foreach  $S_n \in \mathcal{S}$  do
3   |  $\text{sim} \leftarrow \frac{|S_q \cap S_n|}{\sqrt{|S_q| \times |S_n|}}$ ;
4   | if  $\text{sim} > 0$  then
5   |   | Thêm  $(S_n, \text{sim})$  vào similarities;
6   | end
7 end

// Bước 2: Lấy K phiên tương tự nhất
8 Sắp xếp similarities theo sim giảm dần;
9  $KNN \leftarrow K$  phần tử đầu tiên của similarities;

// Bước 3: Tính score cho mỗi item ứng viên
10 scores  $\leftarrow \{ \}$ ; // Dictionary rỗng
11 foreach  $(S_n, \text{sim}) \in KNN$  do
12   | foreach  $\text{item } i \in S_n \text{ mà } i \notin S_q$  do
13   |   | scores[ $i$ ]  $\leftarrow$  scores[ $i$ ] + sim;
14   | end
15 end

// Bước 4: Trả về Top-N
16 Sắp xếp scores theo giá trị giảm dần;
17 return  $N$  item đầu tiên

```

2.5.6. Tối ưu hóa bằng chỉ mục đảo (Inverted Index)

Trong thực tế, việc tính similarity với **tất cả** phiên lịch sử là rất tốn kém khi N lớn (hàng triệu phiên). Để tăng tốc, ta sử dụng **inverted index**:

Inverted Index: Với mỗi item i , lưu danh sách các session chứa item đó:

$$\text{index}[i] = \{S_n : i \in S_n\}$$

Khi cần tìm hàng xóm cho $S_q = [i_1, i_2, \dots, i_t]$:

1. Tra cứu $\text{index}[i_k]$ cho mỗi $i_k \in S_q$
2. Hợp các danh sách $\rightarrow$ chỉ xét những session có ít nhất 1 item chung
3. Tính similarity chỉ cho các session ứng viên này

Điều này giảm đáng kể số phép tính, đặc biệt khi phần lớn session không có item chung với S_q .

2.5.7. Phân tích độ phức tạp

Không gian (Space complexity):

- Lưu trữ tất cả phiên lịch sử: $O(N \times \bar{T})$ với $\bar{T}$ là độ dài trung bình session
- Inverted index: $O(M \times \bar{F})$ với $\bar{F}$ là số session trung bình chứa mỗi item

Thời gian (Time complexity):

- Không dùng inverted index: $O(N \times |S_q|)$ – phải duyệt tất cả session
- Dùng inverted index: $O(|S_q| \times \bar{F} + C \times \log C)$ với C là số session ứng viên
- Trong thực tế: $C \ll N$, nên tốc độ cải thiện đáng kể

Ưu điểm của SKNN:

1. Không cần huấn luyện (training-free) – chỉ cần lưu dữ liệu
2. Dễ cập nhật: thêm session mới chỉ cần thêm vào index
3. Giải thích được (interpretable): có thể chỉ ra “tại sao gợi ý item này”
4. Hiệu quả cao trên nhiều dataset thực tế

2.6. Thuật toán GRU4Rec

2.6.1. Mạng hồi quy RNN và vấn đề tiêu biến gradient

Mạng hồi quy (Recurrent Neural Network – RNN) là kiến trúc mạng nơ-ron được thiết kế để xử lý dữ liệu tuần tự [8]. Tại mỗi bước thời gian t , RNN nhận input x_t và hidden state trước đó h_{t-1} , tạo ra hidden state mới:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \quad (2.8)$$

$$y_t = W_{hy}h_t + b_y \quad (2.9)$$

Vấn đề Vanishing/Exploding Gradient: Khi huấn luyện RNN bằng Backpropagation Through Time (BPTT), gradient phải lan truyền ngược qua nhiều bước thời gian. Gradient tại bước t phụ thuộc vào tích của nhiều ma trận Jacobian:

$$\frac{\partial h_t}{\partial h_1} = \prod_{k=2}^t \frac{\partial h_k}{\partial h_{k-1}} = \prod_{k=2}^t W_{hh}^T \cdot \text{diag}(\tanh'(z_k)) \quad (2.10)$$

Nếu eigenvalue lớn nhất của W_{hh} nhỏ hơn 1, gradient sẽ **tiêu biến** (vanishing) theo cấp số nhân khi t tăng. Ngược lại, nếu lớn hơn 1, gradient sẽ **bùng nổ** (exploding). Điều này khiến RNN cơ bản không thể học được phụ thuộc dài hạn (long-term dependencies).

2.6.2. Gated Recurrent Unit (GRU)

GRU được đề xuất bởi Cho et al. (2014) [5] như một giải pháp cho vấn đề vanishing gradient, đồng thời đơn giản hơn LSTM (Long Short-Term Memory) [11]. GRU sử dụng hai cổng (gate) để kiểm soát luồng thông tin:

Công thức chi tiết của GRU:

Tại mỗi bước thời gian t , với input $x_t \in \mathbb{R}^d$ và hidden state trước đó $h_{t-1} \in \mathbb{R}^H$:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad (\text{Update gate}) \quad (2.11)$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad (\text{Reset gate}) \quad (2.12)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h) \quad (\text{Candidate hidden state}) \quad (2.13)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (\text{New hidden state}) \quad (2.14)$$

Giải thích chi tiết từng thành phần:

Bảng 2.5: Giải thích các thành phần trong GRU

Ký hiệu	Tên	Ý nghĩa
$x_t \in \mathbb{R}^d$	Input	Vector embedding của item tại bước t
$h_{t-1} \in \mathbb{R}^H$	Previous state	Trạng thái ẩn từ bước trước
$z_t \in (0, 1)^H$	Update gate	Quyết định bao nhiêu thông tin cũ được giữ lại
$r_t \in (0, 1)^H$	Reset gate	Quyết định bao nhiêu thông tin cũ ảnh hưởng đến candidate
$\tilde{h}_t \in (-1, 1)^H$	Candidate	Trạng thái ẩn ứng viên (thông tin mới)
$h_t \in \mathbb{R}^H$	New state	Trạng thái ẩn mới (kết hợp cũ và mới)
$\sigma(\cdot)$	Sigmoid	Hàm kích hoạt, output $\in (0, 1)$
$\tanh(\cdot)$	Tanh	Hàm kích hoạt, output $\in (-1, 1)$
$\odot$	Hadamard product	Phép nhân từng phần tử (element-wise)
$W_z, W_r, W_h \in \mathbb{R}^{H \times d}$	Input weights	Ma trận trọng số cho input
$U_z, U_r, U_h \in \mathbb{R}^{H \times H}$	Recurrent weights	Ma trận trọng số cho hidden state
$b_z, b_r, b_h \in \mathbb{R}^H$	Biases	Vector bias

Cơ chế hoạt động trực quan:

1. **Update gate z_t :** Khi $z_t \approx 0$, hidden state mới gần bằng h_{t-1} (giữ nguyên thông tin cũ, “nhớ” quá khứ). Khi $z_t \approx 1$, hidden state mới gần bằng $\tilde{h}_t$ (thay thế bằng thông tin mới, “quên” quá khứ).
2. **Reset gate r_t :** Khi $r_t \approx 0$, candidate $\tilde{h}_t$ được tính chỉ từ input x_t (bỏ qua quá khứ hoàn toàn). Khi $r_t \approx 1$, quá khứ h_{t-1} ảnh hưởng đầy đủ đến candidate.
3. **Tại sao giải quyết vanishing gradient:** Công thức $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$ tạo ra một “đường tắt” (shortcut) cho gradient. Khi $z_t \approx 0$, gradient có thể chảy trực tiếp từ h_t về h_{t-1} mà không bị nhân với ma trận trọng số, tránh vanishing.

2.6.3. Kiến trúc GRU4Rec

GRU4Rec (Hidasi et al., 2016) [1] áp dụng GRU cho bài toán session-based recommendation với kiến trúc:

$$i_t \xrightarrow{\text{Embedding}} e_{i_t} \xrightarrow{\text{GRU}} h_t \xrightarrow{\text{Linear}} \hat{y}_t \in \mathbb{R}^M \xrightarrow{\text{Softmax}} P(i_{t+1}) \quad (2.15)$$

Chi tiết từng lớp:**1. Embedding Layer:**

$$e_{i_t} = E[i_t] \in \mathbb{R}^d \quad (2.16)$$

trong đó $E \in \mathbb{R}^{M \times d}$ là ma trận embedding, M là tổng số item, d là kích thước embedding (thường $d = 50$ hoặc 64). Mỗi item được biểu diễn bằng một vector dense d chiều, được học trong quá trình huấn luyện.

2. GRU Layer:

$$h_t = \text{GRU}(e_{i_t}, h_{t-1}) \quad (2.17)$$

Áp dụng các công thức (2.11)–(2.14) với $x_t = e_{i_t}$. Hidden state $h_t \in \mathbb{R}^H$ (thường $H = 100$ hoặc 128) mã hóa toàn bộ thông tin từ chuỗi $[i_1, \dots, i_t]$.

3. Output Layer (Linear + Softmax):

$$\hat{y}_t = W_o \cdot h_t + b_o \in \mathbb{R}^M \quad (2.18)$$

$$P(i_{t+1} = j \mid i_1, \dots, i_t) = \frac{\exp(\hat{y}_t[j])}{\sum_{k=1}^M \exp(\hat{y}_t[k])} \quad (2.19)$$

trong đó $W_o \in \mathbb{R}^{M \times H}$ là ma trận trọng số output, $b_o \in \mathbb{R}^M$ là bias. Vector $\hat{y}_t$ chứa logit (điểm số chưa chuẩn hóa) cho tất cả M item. Softmax chuyển logit thành phân phối xác suất.

4. Khuyến nghị:

$$\text{Top-}N = \underset{j \notin S_q}{\text{argmax-}N} \hat{y}_t[j] \quad (2.20)$$

Lấy N item có logit cao nhất (loại bỏ item đã xem trong phiên hiện tại).

2.6.4. Hàm mất mát

a) Cross-Entropy Loss:

Đây là hàm mất mát phổ biến nhất cho bài toán phân loại đa lớp. Trong ngữ cảnh session-based RS, mỗi bước dự đoán là một bài toán phân loại M lớp (chọn 1 trong M item):

$$\mathcal{L}_{CE} = -\frac{1}{|\mathcal{D}|} \sum_{(S, i^+) \in \mathcal{D}} \log P(i^+ \mid S) \quad (2.21)$$

trong đó:

- $\mathcal{D}$: Tập tất cả cặp huấn luyện (input sequence S , target item i^+)
- i^+ : Item thực sự được click tiếp theo (ground truth / positive item)
- $P(i^+|S)$: Xác suất mô hình gán cho item đúng (từ softmax)
- Mục tiêu: Tối đa hóa xác suất của item đúng $\Leftrightarrow$ Tối thiểu hóa $\mathcal{L}_{CE}$

b) BPR Loss (Bayesian Personalized Ranking):

BPR Loss được đề xuất bởi Rendle et al. (2009) [6] dựa trên ý tưởng: thay vì dự đoán xác suất tuyệt đối, chỉ cần đảm bảo item đúng (i^+) có score **cao hơn** item sai (i^-):

$$\mathcal{L}_{BPR} = -\frac{1}{|\mathcal{D}|} \sum_{(S, i^+) \in \mathcal{D}} \frac{1}{N_s} \sum_{j=1}^{N_s} \log \sigma(\hat{y}[i^+] - \hat{y}[j]) \quad (2.22)$$

trong đó:

- j : Negative sample – item được chọn ngẫu nhiên (giả định là item người dùng không quan tâm)
- N_s : Số negative samples cho mỗi positive sample
- $\sigma(x) = \frac{1}{1+e^{-x}}$: Hàm sigmoid
- $\hat{y}[i^+] - \hat{y}[j]$: Chênh lệch score giữa item đúng và item sai
- Khi $\hat{y}[i^+] \gg \hat{y}[j]$: $\sigma(\cdot) \approx 1$, $\log(\cdot) \approx 0$ (loss thấp)
- Khi $\hat{y}[i^+] \ll \hat{y}[j]$: $\sigma(\cdot) \approx 0$, $\log(\cdot) \rightarrow -\infty$ (loss cao)

So sánh hai hàm loss:

Bảng 2.6: So sánh Cross-Entropy Loss và BPR Loss

Tiêu chí	Cross-Entropy	BPR
Mục tiêu	Tối đa xác suất item đúng	Đảm bảo item đúng > item sai
Tính toán	Softmax trên toàn bộ M item (tốn kém khi M lớn)	Chỉ cần so sánh cặp (positive, negative)
Negative sampling	Không cần (implicit qua softmax)	Cần chọn negative samples
Hiệu quả	Tốt khi M không quá lớn	Tốt khi M rất lớn

Trong thí nghiệm của chúng tôi, sử dụng **Cross-Entropy Loss** vì số lượng item sau tiền xử lý không quá lớn (10,435 item, gồm cả token padding).

2.6.5. Mã giả huấn luyện GRU4Rec

Thuật toán 3 trình bày quy trình huấn luyện GRU4Rec.

Thuật toán 3: GRU4Rec – quy trình huấn luyện**Input:** Tập phiên huấn luyện $\mathcal{S}_{train}$, learning rate η , số epoch E , batch size B **Output:** Tham số mô hình $\theta = \{E, W_z, U_z, W_r, U_r, W_h, U_h, W_o, b_o, \dots\}$

```

1 Khởi tạo  $\theta \sim \mathcal{N}(0, 0.01)$ ;
2 for  $epoch = 1$  to  $E$  do
3   Xáo trộn  $\mathcal{S}_{train}$ ;
4   total_loss  $\leftarrow 0$ ;
5   foreach mini-batch  $\mathcal{B}$  gồm  $B$  session do
6     foreach session  $[i_1, i_2, \dots, i_T] \in \mathcal{B}$  do
7        $h_0 \leftarrow \mathbf{0} \in \mathbb{R}^H$ ; // Khởi tạo hidden state
8       for  $t = 1$  to  $T - 1$  do
9          $e_t \leftarrow E[i_t]$ ; // Embedding lookup
10         $h_t \leftarrow \text{GRU}(e_t, h_{t-1})$ ; // Forward GRU
11         $\hat{y}_t \leftarrow W_o \cdot h_t + b_o$ ; // Compute logits
12         $\mathcal{L}_t \leftarrow -\log \frac{\exp(\hat{y}_t[i_{t+1}])}{\sum_k \exp(\hat{y}_t[k])}$ ; // CE Loss
13      end
14    end
15     $\mathcal{L}_{batch} \leftarrow \frac{1}{|\mathcal{B}|} \sum \mathcal{L}_t$ ;
16     $\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \mathcal{L}_{batch}$ ; // Gradient descent (Adam)
17    Clip gradient:  $\|\nabla\| \leq 5.0$ ; // Tránh exploding gradient
18    total_loss  $\leftarrow$  total_loss +  $\mathcal{L}_{batch}$ ;
19  end
20  In ra: “Epoch epoch, Loss = total_loss / num_batches”;
21 end
22 return  $\theta$ 

```

2.6.6. Mã giả dự đoán GRU4Rec

Thuật toán 4 trình bày quy trình dự đoán (inference) của GRU4Rec.

Thuật toán 4: GRU4Rec – quy trình dự đoán**Input:** Phiên hiện tại $S_q = [i_1, \dots, i_t]$, Mô hình đã huấn luyện, Số khuyến nghị N **Output:** Top- N item khuyến nghị

```

1  $h_0 \leftarrow \mathbf{0} \in \mathbb{R}^H$ ;
2 for  $k = 1$  to  $t$  do
3    $e_k \leftarrow E[i_k]$  ; // Embedding lookup
4    $h_k \leftarrow \text{GRU}(e_k, h_{k-1})$  ; // Forward GRU
5 end

// Tính score cho tất cả item
6  $\hat{y} \leftarrow W_o \cdot h_t + b_o \in \mathbb{R}^M$ ;

// Loại bỏ item đã xem
7 foreach  $i \in S_q$  do
8    $\hat{y}[i] \leftarrow -\infty$ ;
9 end

// Lấy Top-N
10  $\text{top\_indices} \leftarrow \text{argsort}(\hat{y}, \text{descending})[: N]$ ;
11 return  $\text{top\_indices}$ 

```

2.6.7. Phân tích độ phức tạp GRU4Rec**Số tham số mô hình:**

- Embedding layer: $M \times d$ tham số
- GRU layer: $3(d \times H + H \times H) + 6H$ tham số (3 cổng, mỗi cổng có ma trận W_{ih} , W_{hh} và hai vector bias theo cài đặt PyTorch)
- Output layer: $H \times M + M$ tham số
- **Tổng:** $M(d + H) + 3(dH + H^2) + 6H + M$ (với $M \gg d, H$, chi phối bởi embedding Md và output MH)

Với cấu hình thí nghiệm ($M = 10,435$ – gồm cả token padding, $d = 64$, $H = 128$):

$$\text{Tổng tham số} = \underbrace{667,840}_{\text{Embedding}} + \underbrace{74,496}_{\text{GRU}} + \underbrace{1,346,115}_{\text{Output}} = 2,088,451 \quad (2.23)$$

Thời gian huấn luyện:

$$O(E \times |\mathcal{S}| \times \bar{T} \times (d \cdot H + H^2 + H \cdot M)) \quad (2.24)$$

trong đó E là số epoch, $|\mathcal{S}|$ là số session, $\bar{T}$ là độ dài trung bình session.

Thời gian inference (cho 1 session):

$$O(t \times (d \cdot H + H^2) + H \times M) \quad (2.25)$$

Với t nhỏ (3–5 item) và M vừa phải, inference rất nhanh (vài ms), phù hợp cho ứng dụng real-time.

2.7. Các chỉ số đánh giá

Trong bài toán session-based recommendation, ta đánh giá khả năng của mô hình trong việc đặt item đúng (ground truth) vào danh sách Top- K khuyến nghị. Các chỉ số phổ biến:

2.7.1. Recall@K (Hit Rate)

Recall@K đo tỷ lệ các trường hợp test mà item đúng nằm trong Top- K khuyến nghị:

$$\text{Recall@K} = \frac{1}{|\mathcal{T}|} \sum_{(S, i^+) \in \mathcal{T}} \mathbb{I}[i^+ \in \text{Top-}K(S)] \quad (2.26)$$

trong đó:

- $\mathcal{T}$: Tập test (các cặp input session S và target item i^+)
- $\text{Top-}K(S)$: Danh sách K item được mô hình khuyến nghị cho session S
- $\mathbb{I}[\cdot]$: Hàm chỉ thị, bằng 1 nếu điều kiện đúng
- Giá trị $\in [0, 1]$: càng cao càng tốt

Ý nghĩa: “Trong bao nhiêu phần trăm trường hợp, item mà người dùng thực sự click tiếp theo có nằm trong Top- K gợi ý của mô hình?”

2.7.2. MRR@K (Mean Reciprocal Rank)

MRR@K không chỉ quan tâm item đúng có trong Top- K hay không, mà còn quan tâm **vị trí** của nó:

$$\text{MRR@K} = \frac{1}{|\mathcal{T}|} \sum_{(S, i^+) \in \mathcal{T}} \begin{cases} \frac{1}{\text{rank}(i^+)} & \text{nếu } \text{rank}(i^+) \leq K \\ 0 & \text{nếu } \text{rank}(i^+) > K \end{cases} \quad (2.27)$$

trong đó $\text{rank}(i^+)$ là vị trí của item đúng trong danh sách khuyến nghị (1-indexed).

Ý nghĩa: MRR thưởng cho mô hình đặt item đúng ở vị trí cao (rank 1 $\rightarrow$ score 1.0, rank 2 $\rightarrow$ 0.5, rank 3 $\rightarrow$ 0.33, ...). Chỉ số này phản ánh chất lượng xếp hạng tốt hơn Recall.

2.7.3. HR@K (Hit Rate)

Hit Rate@K tương đương với Recall@K trong trường hợp mỗi test case chỉ có 1 item đúng (đúng với bài toán session-based RS):

$$\text{HR@K} = \text{Recall@K} = \frac{\text{Số test case có hit}}{|\mathcal{T}|} \quad (2.28)$$

2.7.4. NDCG@K (Normalized Discounted Cumulative Gain)

NDCG@K là chỉ số đánh giá xếp hạng phổ biến, sử dụng hàm discount logarithmic:

$$\text{DCG@K} = \sum_{k=1}^K \frac{\text{rel}_k}{\log_2(k+1)} \quad (2.29)$$

$$\text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}} \quad (2.30)$$

Trong trường hợp chỉ có 1 item relevant (session-based RS):

$$\text{NDCG@K} = \begin{cases} \frac{1}{\log_2(\text{rank}(i^+)+1)} & \text{nếu } \text{rank}(i^+) \leq K \\ 0 & \text{nếu } \text{rank}(i^+) > K \end{cases} \quad (2.31)$$

So sánh các chỉ số:

Bảng 2.7: So sánh các chỉ số đánh giá

Chỉ số	Quan tâm	Ưu điểm	Phạm vi
Recall@K	Có/không trong Top-K	Đơn giản, dễ hiểu	[0, 1]
MRR@K	Vị trí trong Top-K	Phân biệt rank 1 vs rank K	[0, 1]
NDCG@K	Vị trí (log discount)	Chuẩn hóa, phổ biến	[0, 1]

Trong báo cáo này, chúng tôi sử dụng **Recall@20** và **MRR@20** làm chỉ số chính, phù hợp với các nghiên cứu trước đó [1, 2].

3. THÍ NGHIỆM

3.1. Bộ dữ liệu Yoochoose (RecSys Challenge 2015)

3.1.1. Giới thiệu bộ dữ liệu

Yoochoose là bộ dữ liệu được sử dụng trong RecSys Challenge 2015, chứa dữ liệu click-stream từ một trang web thương mại điện tử châu Âu trong khoảng thời gian 6 tháng (từ tháng 4 đến tháng 9 năm 2014) [14]. Đây là một benchmark phổ biến cho bài toán session-based recommendation, được sử dụng trong nhiều nghiên cứu quan trọng [1, 2, 3, 4, 7].

3.1.2. Cấu trúc dữ liệu

Dataset bao gồm hai file chính:

1. yoochoose-clicks.dat: Chứa tất cả sự kiện click với các trường:

- **Session ID:** Mã phiên (integer)
- **Timestamp:** Thời điểm click (ISO 8601 format)
- **Item ID:** Mã sản phẩm (integer)
- **Category:** Danh mục sản phẩm (string)

2. yoochoose-buys.dat: Chứa các sự kiện mua hàng (subset của clicks).

Trong thí nghiệm này, chúng tôi chỉ sử dụng file `yoochoose-clicks.dat` vì bài toán tập trung vào dự đoán click tiếp theo.

3.1.3. Thống kê tổng quan

Bảng 3.1: Thống kê bộ dữ liệu Yoochoose

Thuộc tính	Toàn bộ dataset	Sau lấy mẫu 1/64
Tổng số sự kiện click	33,003,944	515,686
Số phiên (session)	9,249,729	144,527
Số sản phẩm (item) duy nhất	52,739	22,718
Thời gian thu thập	6 tháng	6 tháng
Lĩnh vực	E-commerce	E-commerce
<i>Sau tiền xử lý (chia theo thời gian, lọc item hiếm trên train, lọc độ dài [2, 20]):</i>		
Số phiên còn lại (train + test)	–	116,267
Số item còn lại (vocab, gồm padding)	–	10,435
Độ dài session trung bình	–	3.7
Độ dài session trung vị	–	3
<i>Chia train/test theo thời gian (90/10):</i>		
Số phiên train	–	108,263
Số phiên test	–	8,004

3.1.4. Lý do lấy mẫu 1/64

Do dataset gốc rất lớn (33 triệu click, 9.2 triệu session), việc chạy thí nghiệm trên toàn bộ dữ liệu đòi hỏi tài nguyên tính toán lớn và thời gian dài. Theo phương pháp của Hidasi et al. (2016) [1], chúng tôi lấy mẫu ngẫu nhiên 1/64 số session để thí nghiệm. Tỷ lệ này đủ lớn để đảm bảo tính đại diện thống kê, đồng thời cho phép chạy nhiều thí nghiệm với các cấu hình khác nhau trong thời gian hợp lý.

3.2. Cài đặt thí nghiệm

3.2.1. Siêu tham số (hyperparameters)

Bảng 3.2: Cấu hình hyperparameters cho các mô hình

Mô hình	Hyperparameter	Giá trị
Popularity Baseline	Top-N	20
	Metric	Tần suất xuất hiện
SKNN	K (số hàng xóm)	500
	Similarity metric	Cosine (binary)
	Top-N	20
GRU4Rec	Embedding size (d)	64
	Hidden size (H)	128
	Số lớp GRU	1
	Dropout	0.25
	Optimizer	Adam
	Learning rate (η)	0.001
	Weight decay	10^{-5}
	Batch size	256
	Số epoch	10
	Gradient clipping	5.0
	Loss function	Cross-Entropy
	Max sequence length	20

3.2.2. Phần cứng và phần mềm

Bảng 3.3: Môi trường thí nghiệm

Thành phần	Chi tiết
Hệ điều hành	Windows 10/11
CPU	Intel Core i5/i7 hoặc tương đương
RAM	8–16 GB
GPU	NVIDIA (nếu có) hoặc CPU-only
Python	3.8+
PyTorch	2.0+
NumPy	1.24+
Pandas	2.0+
Scikit-learn	1.3+
Matplotlib	3.7+

3.3. Tiền xử lý cụ thể

Quy trình tiền xử lý được thực hiện theo các bước sau, với số liệu cụ thể tại mỗi bước:

Bảng 3.4: Số liệu qua từng bước tiền xử lý

Bước	Thao tác	Phiên (train / test)	Số item
0	Mẫu 1/64 ngẫu nhiên (seed=42)	144,527 (chưa chia)	22,718
1	Chia theo thời gian (90/10)	130,074 / 14,453	–
2	Lọc item hiếm (< 5 lần <i>trên train</i>)	130,074 / 14,453	10,449 giữ lại
3	Lọc độ dài [2, 20] + bỏ item hiếm	108,263 / 8,004	10,435 (vocab)

Chi tiết từng bước:

- Lấy mẫu 1/64:** Chọn ngẫu nhiên 1/64 số Session ID từ dataset gốc (seed=42 để tái tạo được kết quả).
- Sắp xếp:** Sắp xếp dữ liệu theo (SessionID, Timestamp) để đảm bảo thứ tự click đúng.
- Nhóm theo session:** Chuyển từ event log sang chuỗi item cho mỗi session.
- Chia train/test theo thời gian (TRƯỚC khi lọc):** Sắp xếp các phiên theo thời điểm bắt đầu, lấy 90% phiên *sớm nhất* làm train (130,074 phiên), 10% phiên

muộn nhất làm test (14,453 phiên). Mốc chia: 2014-09-12 11:29:40 UTC. Việc chia theo thời gian mô phỏng đúng kịch bản thực tế “dùng quá khứ dự đoán tương lai” và tránh data leakage.

5. **Lọc item hiếm (chỉ trên train):** Đếm tần suất mỗi item **chỉ trên tập train**, loại bỏ item xuất hiện dưới 5 lần (giữ 10,449 item). Việc chỉ dùng train để quyết định giữ item nào đảm bảo *không nhìn vào test* – một nguồn data leakage tinh vi thường bị bỏ qua.
6. **Lọc độ dài phiên [2, 20]:** Sau khi bỏ item hiếm khỏi mọi phiên, giữ lại các phiên có từ 2 đến 20 item. Loại phiên 1 item (không tạo được cặp input-label) và phiên quá dài > 20 item (có thể là bot/outlier). Kết quả cuối: train 108,263 phiên, test 8,004 phiên, vocab 10,435 (gồm token padding).

3.4. Giao thức đánh giá và tính hợp lệ của kết quả

Giao thức đánh giá (evaluation protocol). Cả ba mô hình được đánh giá theo cùng một quy trình để đảm bảo so sánh công bằng:

- **Next-item theo kiểu leave-one-out:** với mỗi phiên test $[i_1, \dots, i_n]$, dùng prefix $[i_1, \dots, i_{n-1}]$ làm đầu vào. Item cuối i_n là nhãn cần dự đoán.
- **Loại item đã xem (exclude-seen):** khi xếp hạng Top-20, mọi item đã có trong đầu vào đều bị loại, vì mục tiêu là dự đoán item *tiếp theo* chứ không nhắc lại item cũ.
- **Cùng metric, cùng K:** Recall@20 và MRR@20 cho cả ba mô hình.
- **Đánh giá trên toàn bộ tập test:** 8,004 phiên (GRU4Rec sinh dự đoán hợp lệ cho 8,003 phiên – 1 phiên có toàn bộ item nằm ngoài từ vựng train nên bị bỏ qua). Không lấy mẫu con tập test.
- **Tính tái lập:** cố định seed=42 cho cả việc lấy mẫu dữ liệu và khởi tạo trọng số GRU4Rec.

Vì sao các chỉ số thấp hơn con số trong bài báo gốc? Hidasi et al. (2016) [1] báo cáo Recall@20 cao hơn nhiều trên *toàn bộ* Yoochoose. Con số của chúng tôi thấp hơn là điều *đã được dự đoán trước*, không phải lỗi, do hai khác biệt về thiết kế thí nghiệm:

1. **Chỉ dùng 1/64 dữ liệu:** ít hơn 64 lần số phiên huấn luyện $\rightarrow$ inverted index của SKNN thưa hơn và GRU4Rec có ít dữ liệu hơn để học embedding, nên Recall tuyệt đối giảm theo.

2. **Chia train/test theo thời gian (không phải ngẫu nhiên):** tập test gồm các phiên *muộn nhất*, chứa nhiều item/hành vi chưa từng thấy trong train. Đây là kịch bản khó hơn nhưng *trung thực* hơn so với chia ngẫu nhiên – vốn gây rò rỉ thông tin tương lai (data leakage) và thổi phồng kết quả.

Điều quan trọng về mặt học thuật là *thứ hạng tương đối* giữa các mô hình (Popularity $\ll$ GRU4Rec $\lesssim$ SKNN), và thứ hạng này nhất quán với tài liệu [2]. Mọi con số trong báo cáo được sinh tự động từ một lần chạy pipeline (`run_all.py`) và lưu trong `output/all_results.pkl`; không có số nào được điền tay.

3.5. Kết quả mô hình Popularity Baseline

Popularity Baseline là phương pháp đơn giản nhất: gợi ý Top- N item phổ biến nhất (xuất hiện nhiều nhất trong training set), **bất kể** phiên hiện tại chứa gì.

Bảng 3.5: Kết quả Popularity Baseline

Mô hình	Recall@20	MRR@20
Popularity Baseline	0.0187 (1.87%)	0.0041

Nhận xét:

- Recall@20 = 1.87%: Chỉ trong 1.87% trường hợp, item mà người dùng thực sự click tiếp theo nằm trong Top-20 item phổ biến nhất.
- MRR@20 = 0.0041: Khi có hit, item đúng thường ở vị trí rất thấp trong danh sách.
- Kết quả này cho thấy: **chỉ dựa vào popularity là hoàn toàn không đủ**. Cần tận dụng thông tin từ phiên hiện tại.
- Tuy nhiên, Popularity Baseline vẫn quan trọng như một **lower bound** để đánh giá các phương pháp phức tạp hơn.

3.6. Kết quả SKNN (K=500)

Bảng 3.6: Kết quả SKNN với K=500

Mô hình	Recall@20	MRR@20
SKNN (K=500)	0.2952 (29.52%)	0.1349

Nhận xét:

- $\text{Recall@20} = 29.52\%$: Trong gần 1/3 trường hợp, item đúng nằm trong Top-20 gợi ý. Đây là cải thiện **+27.65 điểm** so với Popularity Baseline.
- $\text{MRR@20} = 0.1349$: Trung bình, item đúng nằm ở khoảng vị trí thứ 7 ($1/0.1349 \approx 7.4$) trong danh sách Top-20.
- SKNN đạt hiệu quả cao mặc dù **không cần huấn luyện** (training-free), không cần GPU, và rất đơn giản về mặt triển khai.
- Thời gian inference: khoảng 50–100ms cho mỗi session (với inverted index).

3.7. Kết quả GRU4Rec (10 epoch)

Bảng 3.7: Kết quả GRU4Rec sau 10 epoch huấn luyện

Mô hình	Recall@20	MRR@20
GRU4Rec (10 epoch)	0.2796 (27.96%)	0.1262

Nhận xét:

- $\text{Recall@20} = 27.96\%$: Cải thiện **+26.09 điểm** so với Popularity, nhưng **thấp hơn SKNN 1.56 điểm**.
- $\text{MRR@20} = 0.1262$: Thấp hơn SKNN 0.0087 (SKNN xếp hạng item đúng nhỉnh hơn một chút).
- Trên cấu hình thí nghiệm này (mẫu 1/64, chia theo thời gian, 10 epoch), GRU4Rec **chưa vượt** SKNN. Kết quả này phù hợp với Ludewig & Jannach (2018) [2]: SKNN là baseline cực mạnh mà nhiều mô hình deep learning khó vượt qua, nhất là khi dữ liệu hạn chế và số epoch còn thấp.
- GRU4Rec vẫn có lợi thế lý thuyết (**học được thứ tự click**) và còn nhiều dư địa cải thiện (thêm dữ liệu, thêm epoch, tinh chỉnh siêu tham số) – phân tích chi tiết ở Mục 4.

3.8. Thử nghiệm K khác nhau cho SKNN

Để hiểu ảnh hưởng của hyperparameter K (số hàng xóm) đến hiệu năng SKNN, chúng tôi thử nghiệm với các giá trị $K \in \{50, 100, 200, 500\}$.

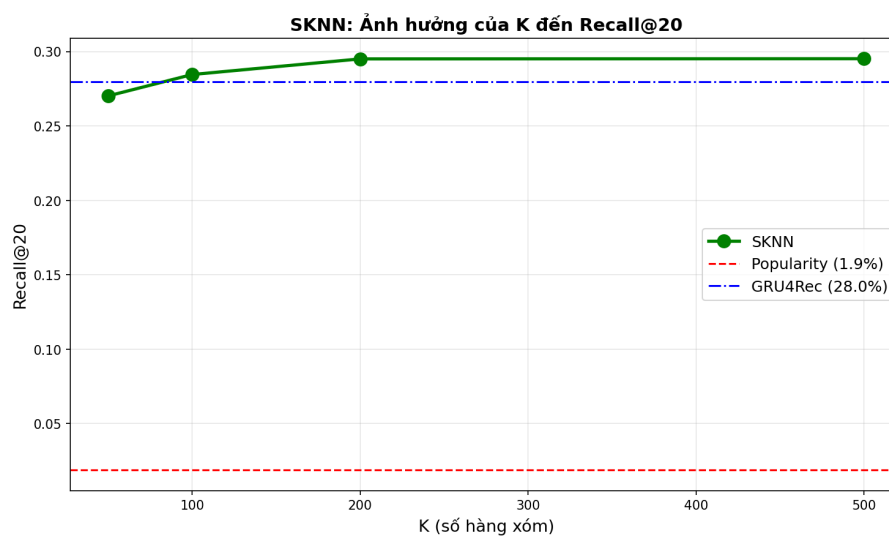
Bảng 3.8: Ảnh hưởng của K đến hiệu năng SKNN

K	Recall@20
50	0.2702
100	0.2846
200	0.2951
500	0.2952

Phân tích:

1. **K=50 (quá nhỏ):** Chỉ xét 50 phiên tương tự nhất. Nhiều item tốt bị bỏ sót vì không đủ “phiếu bầu” từ hàng xóm. Recall thấp nhất (0.2702).
2. **K=100 và K=200:** Hiệu năng tăng đều so với K=50 (0.2846 rồi 0.2951). Đây là giai đoạn thêm hàng xóm vẫn còn mang lại lợi ích rõ rệt.
3. **K=500 (tốt nhất):** Đạt Recall cao nhất (0.2952), nhưng chỉ nhỉnh hơn K=200 đúng 0.0001 – tức đã gần như bão hòa. Với K lớn, mô hình “nhìn thấy” nhiều pattern hơn nhưng lợi ích biên giảm dần (diminishing returns).
4. **Trade-off:** K lớn hơn $\rightarrow$ Recall cao hơn nhưng thời gian tính toán cũng tăng. Từ K=200 trở lên, lợi ích gần như không đáng kể nên K=200–500 là vùng cân bằng hợp lý.

Hình 3.1 minh họa xu hướng Recall@20 theo K.



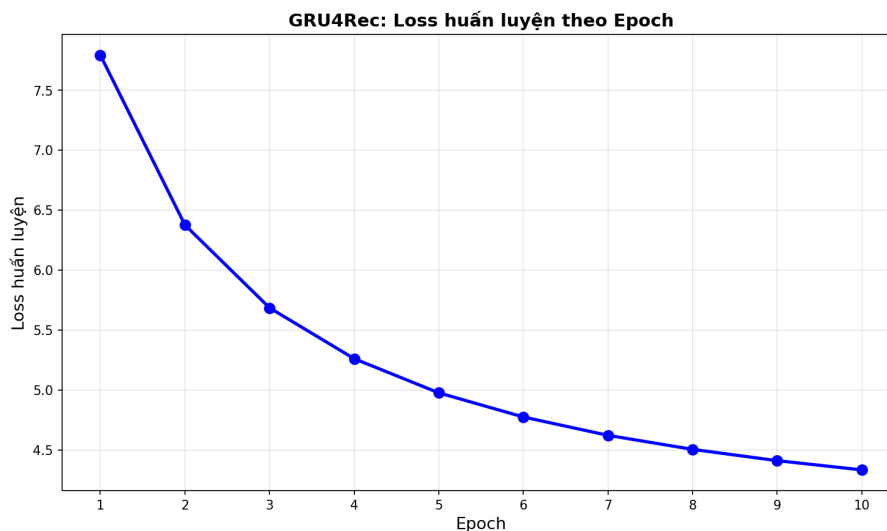
Hình 3.1: Ảnh hưởng của K đến Recall@20 của SKNN

Nhận xét từ biểu đồ:

- Recall tăng nhanh từ $K=50$ đến $K=200$ (giai đoạn “thiếu hàng xóm”)
- Từ $K=200$ đến $K=500$, đường cong gần như nằm ngang ($0.2951 \rightarrow 0.2952$) – đã bão hòa (diminishing returns)
- Nếu tiếp tục tăng K (>1000), có thể Recall sẽ giảm do nhiễu từ các phiên kém tương tự
- $K=200-500$ là vùng cân bằng tốt cho dataset này

3.9. Đường cong loss huấn luyện của GRU4Rec

Hình 3.2 thể hiện đường cong loss trong quá trình huấn luyện GRU4Rec qua 10 epoch.



Hình 3.2: Đường cong loss huấn luyện của GRU4Rec qua 10 epoch

Phân tích đường cong loss:

1. **Epoch 1–3:** Loss giảm nhanh từ 7.79 (trung bình epoch 1; mức khởi tạo lý thuyết $\ln(10435) \approx 9.25$) xuống 5.69 (epoch 3). Đây là giai đoạn mô hình học được các pattern cơ bản nhất (item phổ biến, co-occurrence đơn giản).
2. **Epoch 4–7:** Loss tiếp tục giảm nhưng chậm hơn (từ 5.26 xuống 4.62). Mô hình bắt đầu học các pattern phức tạp hơn (sequential dependencies).
3. **Epoch 8–10:** Loss giảm chậm dần ($4.51 \rightarrow 4.33$), chưa hoàn toàn hội tụ – vẫn còn xu hướng giảm nhẹ.
4. **Không có overfitting rõ ràng:** Loss không tăng trở lại, cho thấy mô hình chưa overfit trên training set. Có thể huấn luyện thêm epoch để cải thiện.

Gợi ý cải thiện:

- Tăng số epoch lên 20–30 để mô hình hội tụ tốt hơn
- Sử dụng learning rate scheduling (giảm lr khi loss bão hòa)
- Tăng hidden size (256 hoặc 512) để tăng capacity
- Thêm early stopping dựa trên validation loss

4. PHÂN TÍCH VÀ THẢO LUẬN

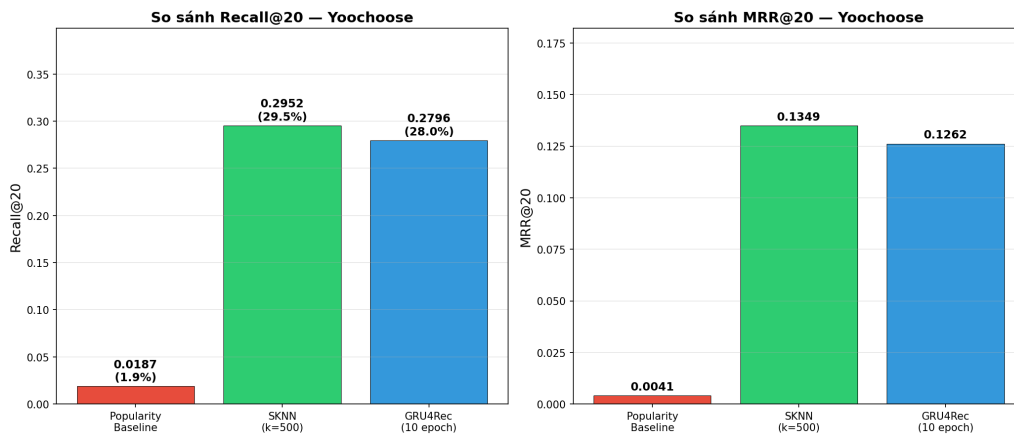
4.1. So sánh tổng hợp 3 mô hình

Bảng 4.1 tổng hợp kết quả của cả ba mô hình trên bộ dữ liệu Yoochoose (1/64).

Bảng 4.1: Bảng so sánh tổng hợp kết quả 3 mô hình

Mô hình	Recall@20	MRR@20	Δ Recall	Thời gian	GPU
Popularity Baseline	0.0187	0.0041	—	~5s	Không
SKNN (K=500)	0.2952	0.1349	+0.2765	~2s	Không
GRU4Rec (10 epoch)	0.2796	0.1262	+0.2609	~3 phút	Có

Hình 4.1 minh họa trực quan sự khác biệt giữa 3 mô hình.



Hình 4.1: Biểu đồ so sánh Recall@20 và MRR@20 của 3 mô hình

Nhận xét tổng quan:

1. Cả SKNN và GRU4Rec đều vượt trội hoàn toàn so với Popularity Baseline (cải thiện hơn 26 điểm Recall).
2. SKNN nhỉnh hơn GRU4Rec một chút (+1.56 điểm Recall, +0.0087 MRR) trên cấu hình thí nghiệm này.
3. Việc SKNN ngang ngửa hoặc vượt GRU4Rec là kết quả thường gặp, phù hợp với các nghiên cứu trước [2, 3].

4.2. Phân tích tại sao SKNN vượt Popularity (+27.65 điểm)

Sự vượt trội lớn của SKNN so với Popularity Baseline (+27.65 điểm Recall, tức gấp ~15.8 lần) có thể được giải thích bởi các yếu tố sau:

4.2.1. Tận dụng thông tin phiên hiện tại

Popularity Baseline hoàn toàn **bỏ qua** thông tin từ phiên hiện tại – nó gợi ý cùng một danh sách item cho mọi người dùng, bất kể họ đang xem gì. Ngược lại, SKNN sử dụng phiên hiện tại để tìm các phiên tương tự, từ đó sinh khuyến nghị **cá nhân hóa theo ngữ cảnh**.

Ví dụ minh họa:

- Phiên hiện tại: [Laptop, Chuột, Bàn phím]
- Popularity gợi ý: [iPhone, Áo thun, Giày] (item phổ biến nhất, không liên quan)
- SKNN tìm phiên tương tự: [Laptop, Chuột, Bàn phím, **Tai nghe**]
- SKNN gợi ý: [**Tai nghe**, Webcam, USB Hub] (liên quan đến ngữ cảnh)

4.2.2. Khai thác co-occurrence pattern

SKNN ngầm khai thác mối quan hệ đồng xuất hiện (co-occurrence) giữa các item. Nếu item A và item B thường xuất hiện cùng nhau trong nhiều session, thì khi phiên hiện tại chứa A, SKNN sẽ có xu hướng gợi ý B (vì nhiều phiên hàng xóm chứa cả A và B).

4.2.3. Phân tích định lượng

Bảng 4.2: Phân tích chi tiết sự cải thiện của SKNN so với Popularity

Chỉ số	Popularity	SKNN
Recall@20	0.0187	0.2952
Cải thiện tuyệt đối	–	+0.2765
Cải thiện tương đối	–	×15.8 lần
MRR@20	0.0041	0.1349
Cải thiện tuyệt đối	–	+0.1308
Cải thiện tương đối	–	×32.9 lần

MRR cải thiện mạnh hơn Recall (×32.9 vs ×15.8), cho thấy SKNN không chỉ tìm đúng item mà còn đặt item đúng ở vị trí cao hơn nhiều trong danh sách.

4.3. Phân tích tại sao GRU4Rec chưa vượt SKNN

Trên cấu hình thí nghiệm này, GRU4Rec thấp hơn SKNN 1.56 điểm Recall. Tuy nhiên, về *lý thuyết* GRU4Rec vẫn có những ưu thế quan trọng mà điều kiện thí nghiệm hạn chế (mẫu 1/64, 10 epoch) chưa cho phép phát huy:

4.3.1. Học được thứ tự (Sequential patterns)

SKNN coi mỗi session như một **tập hợp** item (bỏ qua thứ tự). GRU4Rec coi session như một **chuỗi** có thứ tự và học được rằng:

- Chuỗi $[A \rightarrow B \rightarrow C]$ có ý nghĩa khác với $[C \rightarrow B \rightarrow A]$
- Item gần đây (cuối chuỗi) có ảnh hưởng lớn hơn item xa (đầu chuỗi)
- Có những transition pattern: “sau khi xem A thường xem B” mà SKNN không nắm bắt được

4.3.2. Biểu diễn liên tục (Continuous representation)

GRU4Rec biểu diễn mỗi item bằng vector embedding dense trong không gian liên tục $\mathbb{R}^d$. Điều này cho phép:

- Nắm bắt sự tương tự ngữ nghĩa giữa các item (item tương tự có embedding gần nhau)
- Tổng quát hóa tốt hơn cho item hiếm (thông qua embedding được chia sẻ)
- Mã hóa thông tin phong phú hơn so với biểu diễn nhị phân (có/không) của SKNN

4.3.3. Tại sao GRU4Rec lại thấp hơn SKNN ở đây?

Có nhiều lý do giải thích tại sao GRU4Rec chưa vượt được SKNN trong thí nghiệm này:

1. **Session ngắn:** Với session trung bình chỉ 3.7 item (trung vị 3), lợi thế “học thứ tự” của GRU bị hạn chế. Thứ tự chỉ thực sự quan trọng khi chuỗi đủ dài.
2. **Dữ liệu hạn chế:** Chỉ dùng mẫu 1/64 ($\sim 108K$ phiên train) – chưa đủ để GRU học tốt embedding cho hơn 10K item. Trên toàn bộ dataset, GRU thường thu hẹp hoặc vượt khoảng cách này.
3. **Chưa hội tụ:** GRU4Rec chỉ được huấn luyện 10 epoch, loss vẫn đang giảm (4.41 $\rightarrow$ 4.33 ở epoch 9–10), chưa hoàn toàn hội tụ. Với nhiều epoch hơn, kết quả nhiều khả năng cải thiện thêm.
4. **SKNN rất mạnh:** Như Ludewig & Jannach (2018) [2] đã chỉ ra, SKNN là baseline cực kỳ mạnh, thường ngang ngửa hoặc vượt nhiều phương pháp deep learning – đặc biệt khi item phổ biến lặp lại giữa các phiên gần nhau (rất đúng với hành vi e-commerce của Yoochoose).

4.4. Ưu nhược điểm từng phương pháp

Bảng 4.3: Phân tích ưu nhược điểm chi tiết

Mô hình	Ưu điểm	Nhược điểm
Popularity	• Cực kỳ đơn giản • Không cần tính toán • Baseline tốt để so sánh • Luôn có kết quả (không fail)	• Không cá nhân hóa • Bỏ qua ngữ cảnh phiên • Hiệu quả rất thấp • Gợi ý giống nhau cho mọi user
SKNN	• Không cần huấn luyện • Không cần GPU • Dễ triển khai, dễ debug • Giải thích được (interpretable) • Dễ cập nhật (thêm session mới) • Hiệu quả cao	• Bỏ qua thứ tự click • Cần lưu toàn bộ session lịch sử • Tốc độ chậm khi N lớn • Không học được biểu diễn item • Khó mở rộng cho dataset rất lớn
GRU4Rec	• Học được thứ tự click • Biểu diễn item liên tục • Inference nhanh (vài ms) • Tổng quát hóa tốt • Có thể mở rộng (scalable)	• Cần huấn luyện (tốn thời gian) • Cần GPU cho training • Khó giải thích (black-box) • Nhiều hyperparameter cần tune • Khó cập nhật incremental • Cần nhiều dữ liệu để học tốt

4.5. Ảnh hưởng của siêu tham số K

Phân tích chi tiết hơn về ảnh hưởng của K trong SKNN:

4.5.1. Cơ chế ảnh hưởng

Hyperparameter K kiểm soát **trade-off giữa precision và coverage**:

- **K nhỏ (precision cao, coverage thấp):** Chỉ xét ít phiên rất tương tự $\rightarrow$ item được gợi ý có độ tin cậy cao nhưng đa dạng thấp. Có thể bỏ sót item tốt từ phiên ít tương tự hơn.
- **K lớn (precision thấp hơn, coverage cao):** Xét nhiều phiên, kể cả phiên ít tương tự $\rightarrow$ đa dạng hơn nhưng có thể bị nhiễu từ phiên không liên quan.

4.5.2. Phân tích theo lý thuyết bias–variance

- **K nhỏ $\rightarrow$ High variance:** Kết quả phụ thuộc nhiều vào vài phiên cụ thể. Nếu phiên hàng xóm “may mắn” chứa item đúng $\rightarrow$ hit. Nếu không $\rightarrow$ miss.
- **K lớn $\rightarrow$ High bias:** Kết quả bị “trung bình hóa” bởi nhiều phiên, có xu hướng gợi ý item phổ biến (giống Popularity). Mất đi tính cá nhân hóa.
- **K tối ưu:** Cân bằng giữa bias và variance. Trong thí nghiệm, $K=500$ cho kết quả tốt nhất trên dataset này.

4.5.3. Khuyến nghị chọn K trong thực tế

1. Bắt đầu với $K=100-500$ (phạm vi thường cho kết quả tốt)
2. Dùng validation set để tune K
3. Xem xét trade-off giữa chất lượng và tốc độ
4. K tối ưu phụ thuộc vào đặc điểm dataset (kích thước, sparsity, diversity)

4.6. Hạn chế của thí nghiệm

Các kết quả trên nên được đọc trong phạm vi thiết kế thí nghiệm, thay vì xem như kết luận tuyệt đối cho mọi hệ thống khuyến nghị:

1. **Mẫu dữ liệu 1/64:** Cách lấy mẫu giúp thí nghiệm chạy được trên máy cá nhân, nhưng số liệu tuyệt đối có thể thay đổi khi dùng toàn bộ Yoochoose. Kết luận đáng tin hơn ở đây là thứ tự tương đối trong cùng cấu hình: Popularity thấp rõ rệt, GRU4Rec và SKNN tốt hơn nhiều.

2. **Một miền dữ liệu:** Yoochoose là clickstream thương mại điện tử. Kết quả chưa tự động suy rộng sang tin tức, nhạc, giáo dục, hoặc dataset tiếng Việt nếu hành vi phiên khác đáng kể.
3. **GRU4Rec mới được huấn luyện ở mức vừa phải:** Mô hình chạy 10 epoch và chưa tune rộng learning rate, dropout, batch size hay loss. Vì vậy việc GRU4Rec thấp hơn SKNN trong thí nghiệm này không phủ định tiềm năng của mô hình tuần tự sâu; nó chỉ phản ánh cấu hình đang xét.
4. **Chưa thử các biến thể mạnh hơn:** Phần thực nghiệm chưa bao gồm V-SKNN, STAN, NARM, SR-GNN, hoặc GRU4Rec với BPR loss. Các biến thể này có thể thay đổi thứ hạng mô hình.
5. **Đánh giá offline:** Recall@20 và MRR@20 đo khả năng dự đoán item cuối trong tập test cố định. Chúng chưa thay thế được A/B testing, nơi các yếu tố như tỷ lệ click, chuyển đổi và trải nghiệm người dùng mới được quan sát trực tiếp.
6. **Chỉ dùng chuỗi click:** Thí nghiệm chưa dùng category, price, thời điểm truy cập, dwell time hoặc thông tin nội dung item. Đây là những tín hiệu có thể giúp mô hình phân biệt ý định phiên tốt hơn.
7. **Giao thức leave-one-out còn hẹp:** Mỗi phiên test chỉ dùng item cuối làm nhãn. Một hệ thống thực tế có thể cần đánh giá ở nhiều vị trí trong phiên hoặc đo chất lượng của cả danh sách gợi ý theo thời gian.

5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Tóm tắt kết quả

Báo cáo này trình bày bài toán Session-based Recommendation trong bối cảnh người dùng ẩn danh, nơi hệ thống phải dự đoán item tiếp theo chỉ từ chuỗi click ngắn của phiên hiện tại.

Các kết quả chính:

1. **Popularity Baseline** ($\text{Recall@20} = 1.87\%$): Kết quả này cho thấy việc chỉ gợi ý item phổ biến không đủ cho bài toán theo phiên, nhưng vẫn cần giữ làm mốc so sánh tối thiểu.
2. **SKNN với $K=500$** ($\text{Recall@20} = 29.52\%$, $\text{MRR@20} = 0.1349$): Phương pháp dựa trên phiên lân cận đạt kết quả tốt nhất trong thí nghiệm, cao hơn Popularity +27.65 điểm Recall@20. Điểm mạnh của SKNN là không cần huấn luyện và dễ kiểm tra bằng trực giác.
3. **GRU4Rec với 10 epoch** ($\text{Recall@20} = 27.96\%$, $\text{MRR@20} = 0.1262$): Mô hình học sâu đạt kết quả gần SKNN nhưng **chưa vượt** (thấp hơn 1.56 điểm Recall) trong cấu hình mẫu 1/64 và 10 epoch. Kết quả này hợp lý vì GRU4Rec thường cần nhiều dữ liệu, nhiều epoch và tune siêu tham số kỹ hơn.

Kết luận quan trọng:

- Thông tin trong phiên hiện tại, dù chỉ gồm vài click, đã cải thiện chất lượng gợi ý rất rõ so với popularity thuần túy.
- SKNN là baseline mạnh và dễ giải thích; vì vậy nên xem nó là mốc so sánh nghiêm túc trước khi kết luận một mô hình học sâu là tốt hơn.
- GRU4Rec có lợi thế khi học thứ tự click, nhưng **không mặc định vượt SKNN** nếu dữ liệu ít, phiên ngắn hoặc quá trình huấn luyện chưa được tối ưu.

5.2. Đóng góp chính

Về mặt học thuật và triển khai, bài làm có các đóng góp sau:

1. **Trình bày lý thuyết có kiểm chứng:** Diễn giải SKNN và GRU4Rec từ công thức, trực giác, mã giả đến độ phức tạp, giúp người đọc theo dõi được vì sao thuật toán sinh ra danh sách gợi ý.

2. **Triển khai thực nghiệm tái lập:** Pipeline xử lý dữ liệu, huấn luyện, đánh giá và vẽ biểu đồ được tách thành package `src/`; notebook chỉ dùng để trình bày và gọi lại logic đã kiểm thử.
3. **So sánh cùng điều kiện:** Ba mô hình được đánh giá trên cùng tập train/test, cùng protocol leave-one-out và cùng hai metric Recall@20, MRR@20.
4. **Phân tích ảnh hưởng của K:** Thí nghiệm với nhiều giá trị K cho thấy chất lượng SKNN tăng rồi bão hòa, từ đó giải thích vì sao chọn K=500 trong cấu hình cuối.
5. **Diễn giải kết quả không thỏa phòng:** Báo cáo nêu rõ SKNN thắng trong cấu hình hiện tại, đồng thời chỉ ra những điều kiện mà GRU4Rec có thể được cải thiện thêm.

5.3. Hướng mở rộng

Nếu tiếp tục phát triển đề tài, các hướng sau là hợp lý nhất vì bám trực tiếp vào hạn chế đã nêu:

5.3.1. NARM – Neural Attentive Recommendation Machine

NARM (Li et al., 2017) [4] mở rộng GRU4Rec bằng cách thêm cơ chế Attention:

$$\alpha_t = \text{softmax}(q^T \sigma(W_1 h_t + W_2 h_T)) \quad (5.1)$$

$$c = \sum_{t=1}^T \alpha_t h_t \quad (5.2)$$

Attention cho phép mô hình “chú ý” đến những bước thời gian quan trọng nhất, thay vì chỉ dựa vào hidden state cuối cùng. Điều này đặc biệt hữu ích khi session dài và có nhiều item không liên quan (noise).

Ưu điểm so với GRU4Rec:

- Nắm bắt được cả local preference (item gần đây) và global preference (toàn bộ session)
- Giải thích được phần nào (attention weights cho biết item nào quan trọng)
- Có thể cải thiện so với GRU4Rec trong các thiết lập được báo cáo bởi Li et al. [4]

5.3.2. SR-GNN – Session-based Recommendation with Graph Neural Networks

SR-GNN (Wu et al., 2019) [7] mô hình hóa session dưới dạng đồ thị có hướng:

- Mỗi item là một node
- Mỗi transition (click từ item A sang item B) là một edge
- Sử dụng Graph Neural Network để học biểu diễn item trong đồ thị phiên

Ưu điểm:

- Nắm bắt được quan hệ phức tạp giữa các item (không chỉ sequential)
- Xử lý tốt trường hợp item lặp lại trong session
- Đạt kết quả cạnh tranh trên nhiều benchmark được báo cáo trong bài gốc [7]

5.3.3. BERT4Rec – Bidirectional Encoder Representations for Recommendation

BERT4Rec áp dụng kiến trúc Transformer (self-attention) cho recommendation [13, 12]:

- Sử dụng masked item prediction (tương tự masked language model trong NLP)
- Bidirectional: xem xét cả context trái và phải
- Multi-head self-attention cho phép nắm bắt nhiều loại quan hệ

Ưu điểm:

- Không bị giới hạn bởi vanishing gradient (không dùng RNN)
- Parallel computation (nhanh hơn RNN khi training)
- Nắm bắt long-range dependencies tốt hơn

5.3.4. Dataset tiếng Việt

Một hướng mở rộng thực tế và có giá trị cao là thu thập và thí nghiệm trên dataset từ các nền tảng thương mại điện tử Việt Nam:

- **Tiki:** Nền tảng e-commerce lớn tại Việt Nam
- **Shopee Vietnam:** Marketplace phổ biến nhất
- **Sendo:** Nền tảng nội địa
- **VnExpress:** Trang tin tức (session-based article recommendation)

Dataset tiếng Việt sẽ giúp:

1. Đánh giá hiệu quả các phương pháp trên hành vi người dùng Việt Nam
2. Khám phá đặc thù riêng (ví dụ: ảnh hưởng của ngày lễ, flash sale)
3. Đóng góp cho cộng đồng nghiên cứu trong nước

5.3.5. Các hướng khác

Bảng 5.1: Tổng hợp các hướng mở rộng

Hướng	Mô tả	Độ khó
NARM	Thêm Attention vào GRU	★ ★ ★
SR-GNN	Session dạng đồ thị + GNN	★ ★ ★★
BERT4Rec	Transformer bidirectional	★ ★ ★★
SASRec	Self-Attention Sequential	★ ★ ★★
Dataset Việt Nam	Thu thập từ Tiki/Shopee	★ ★ ★
Context-aware	Thêm time, category, price	★★
Cross-session	Liên kết nhiều session cùng user	★ ★ ★
Real-time serving	Deploy model lên production	★ ★ ★★

TÀI LIỆU THAM KHẢO

- [1] Hidasi, B., Karatzoglou, A., Baltrunas, L., & Tikk, D. (2016). *Session-based Recommendations with Recurrent Neural Networks*. In Proceedings of the International Conference on Learning Representations (ICLR 2016). arXiv:1511.06939. <https://arxiv.org/abs/1511.06939>
- [2] Ludewig, M., & Jannach, D. (2018). *Evaluation of Session-based Recommendation Algorithms*. User Modeling and User-Adapted Interaction (UMUAI), 28(4-5), 331–390. arXiv:1803.09587. <https://arxiv.org/abs/1803.09587>
- [3] Jannach, D., Ludewig, M., & Lerche, L. (2017). *When Recurrent Neural Networks meet the Neighborhood for Session-Based Recommendation*. In Proceedings of the 11th ACM Conference on Recommender Systems (RecSys 2017), pp. 306–310. DOI: 10.1145/3109859.3109872. <https://doi.org/10.1145/3109859.3109872>
- [4] Li, J., Ren, P., Chen, Z., Ren, Z., Lian, T., & Ma, J. (2017). *Neural Attentive Session-based Recommendation*. In Proceedings of the 2017 ACM Conference on Information and Knowledge Management (CIKM 2017), pp. 1419–1428. DOI: 10.1145/3038912.3052569. arXiv:1711.04725. <https://arxiv.org/abs/1711.04725>
- [5] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation*. In Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP 2014), pp. 1724–1734. arXiv:1406.1078. <https://arxiv.org/abs/1406.1078>
- [6] Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L. (2009). *BPR: Bayesian Personalized Ranking from Implicit Feedback*. In Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (UAI 2009), pp. 452–461. arXiv:1205.2618. <https://arxiv.org/abs/1205.2618>
- [7] Wu, S., Tang, Y., Zhu, Y., Wang, L., Xie, X., & Tan, T. (2019). *Session-based Recommendation with Graph Neural Networks*. In Proceedings of the AAAI Conference on Artificial Intelligence (AAAI 2019), 33(01), 346–353. arXiv:1811.00855. <https://arxiv.org/abs/1811.00855>
- [8] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. <https://www.deeplearningbook.org/>
- [9] Koren, Y., Bell, R., & Volinsky, C. (2009). *Matrix Factorization Techniques for Recommender Systems*. Computer, 42(8), 30–37. DOI: 10.1109/MC.2009.263. <https://doi.org/10.1109/MC.2009.263>

- [10] Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). *Item-based Collaborative Filtering Recommendation Algorithms*. In Proceedings of the 10th International Conference on World Wide Web (WWW 2001), pp. 285–295. DOI: 10.1145/371920.372071. https://files.grouplens.org/papers/www10_sarwar.pdf
- [11] Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, 9(8), 1735–1780. DOI: 10.1162/neco.1997.9.8.1735. <https://www.bioinf.jku.at/publications/older/2604.pdf>
- [12] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention Is All You Need*. In Advances in Neural Information Processing Systems (NeurIPS 2017), pp. 5998–6008. arXiv:1706.03762. <https://arxiv.org/abs/1706.03762>
- [13] Sun, F., Liu, J., Wu, J., Pei, C., Lin, X., Ou, W., & Jiang, P. (2019). *BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer*. arXiv:1904.06690. <https://arxiv.org/abs/1904.06690>
- [14] YOOCHOOSE GmbH. (2015). *RecSys Challenge 2015 Dataset*. Kaggle dataset mirror. <https://www.kaggle.com/datasets/chadgostopp/recsys-challenge-2015>

PHỤ LỤC

Phụ lục chỉ giữ các thông tin cần thiết để tái lập thí nghiệm và đối chiếu phần cài đặt cốt lõi. Mã nguồn đầy đủ nằm trong thư mục `src/` của dự án.

A. Hướng dẫn chạy lại thí nghiệm

Chuẩn bị môi trường và thư viện:

```
1 python -m venv venv
2 venv\Scripts\activate
3 pip install -r requirements.txt
```

Listing P.1: Các lệnh chuẩn bị môi trường

Tải dữ liệu:

- Truy cập nguồn dataset: [Kaggle – RecSys Challenge 2015](#).
- Tải `yoochoose-clicks.dat` và đặt vào thư mục `data/`.

Chạy pipeline:

```
1 python run_all.py
2 python plot_final.py
```

Listing P.2: Các lệnh chạy thí nghiệm và vẽ biểu đồ

Bảng P.1: Các tệp quan trọng để tái lập kết quả

Đường dẫn	Vai trò
<code>src/config.py</code>	Lưu siêu tham số, seed, đường dẫn dữ liệu và đường dẫn kết quả.
<code>src/data.py</code>	Tiền xử lý Yoochoose, chia train/test theo thời gian, lọc item hiếm chỉ trên train.
<code>src/models/sknn.py</code>	Cài đặt SKNN bằng chỉ mục đảo.
<code>src/models/gru4rec.py</code>	Cài đặt GRU4Rec bằng PyTorch.
<code>src/evaluate.py</code>	Tính Recall@20 và MRR@20 theo leave-one-out.
<code>output/all_results.pkl</code>	File kết quả cuối cùng, dùng để vẽ biểu đồ và ghi bảng trong báo cáo.

B. Trích đoạn cốt lõi của SKNN

Trích đoạn dưới đây thể hiện hai ý chính của SKNN: xây dựng chỉ mục đảo `item` -> `session` và chỉ tính similarity trên những phiên có item trùng với phiên truy vấn.

```

1 class SKNN:
2     def __init__(self, k=500):
3         self.k = k
4         self.train_sets = {}
5         self.item2sids = defaultdict(list)
6
7     def fit(self, sessions):
8         self.train_sets = {
9             sid: set(items) for sid, items in sessions.items()
10        }
11        for sid, items in self.train_sets.items():
12            for item in items:
13                self.item2sids[item].append(sid)
14
15    def predict(self, query_session, top_n=20):
16        query_set = set(query_session)
17        candidates = set()
18        for item in query_set:
19            candidates.update(self.item2sids.get(item, ()))
20
21        sims = []
22        for sid in candidates:
23            hist_set = self.train_sets[sid]
24            inter = len(query_set & hist_set)
25            if inter:
26                sim = inter / math.sqrt(len(query_set) * len(
27                    hist_set))
28                sims.append((sid, sim))
29
30        sims.sort(key=lambda x: x[1], reverse=True)
31        scores = defaultdict(float)
32        for sid, sim in sims[:self.k]:
33            for item in self.train_sets[sid]:
34                if item not in query_set:
35                    scores[item] += sim
36
37        return sorted(
38            scores.items(), key=lambda x: x[1], reverse=True
39        )[:top_n]

```

Listing P.3: Trích đoạn SKNN với chỉ mục đảo (src/models/sknn.py)

C. Trích đoạn cốt lõi của GRU4Rec và đánh giá

GRU4Rec mã hóa chuỗi item thành embedding, đưa qua GRU, lấy hidden state cuối cùng rồi sinh score cho toàn bộ item trong từ vựng train.

```

1 class GRU4Rec(nn.Module):
2     def __init__(self, n_items, emb_size=64, hidden_size=128,
3                 n_layers=1, dropout=0.25):
4         super().__init__()
5         self.hidden_size = hidden_size
6         self.embedding = nn.Embedding(n_items, emb_size,
7                                     padding_idx=0)
8         self.gru = nn.GRU(
9             emb_size, hidden_size, num_layers=n_layers,
10            batch_first=True, dropout=dropout if n_layers > 1 else
11            0
12        )
13        self.dropout = nn.Dropout(dropout)
14        self.fc_out = nn.Linear(hidden_size, n_items)
15
16    def forward(self, seq):
17        emb = self.dropout(self.embedding(seq))
18        lengths = (seq != 0).sum(dim=1).cpu()
19        packed = nn.utils.rnn.pack_padded_sequence(
20            emb, lengths, batch_first=True, enforce_sorted=False
21        )
22        gru_out, _ = self.gru(packed)
23        gru_out, _ = nn.utils.rnn.pad_packed_sequence(
24            gru_out, batch_first=True
25        )
26        idx = (lengths - 1).unsqueeze(1).unsqueeze(2)
27        idx = idx.expand(-1, 1, self.hidden_size).to(gru_out.device)
28        h_last = gru_out.gather(1, idx).squeeze(1)
29        return self.fc_out(self.dropout(h_last))

```

Listing P.4: Trích đoạn forward của GRU4Rec (src/models/gru4rec.py)

```

1 def evaluate_gru(model, test_sessions, item2idx,
2                 top_n=20, device='cpu'):
3     model.eval()
4     hits, mrr_sum, n_eval = 0, 0.0, 0
5
6     with torch.no_grad():
7         for items in test_sessions.values():
8             input_seq = items[:-1]
9             label_idx = item2idx.get(items[-1], 0)
10            if label_idx == 0:
11                continue
12
13            seq_idx = [item2idx.get(i, 0) for i in input_seq]
14            seq_t = torch.LongTensor([seq_idx]).to(device)

```

```
15         logits = model(seq_t).squeeze(0)
16
17         logits[0] = -float('inf')
18         for seen in seq_idx:
19             if seen > 0:
20                 logits[seen] = -float('inf')
21
22         top_indices = logits.topk(top_n).indices.tolist()
23         if label_idx in top_indices:
24             hits += 1
25             rank = top_indices.index(label_idx) + 1
26             mrr_sum += 1.0 / rank
27         n_eval += 1
28
29     return hits / n_eval, mrr_sum / n_eval
```

Listing P.5: Trích đoạn đánh giá leave-one-out